ERP System — Complete Backend Architecture Specification

Stack: Node.js · Express.js · Sequelize ORM · MySQL 8 · JWT · Socket.io · ExcelJS · Multer · PDFKit

Phase: Pre-code design — no implementation yet

Covers: All 17 modules · 4 roles · Production-grade patterns

---

1. Complete Folder Structure

```
erp-backend/
 |
 ├──── src/
 |    ├──── config/                        # Environment & library bootstrap
 |    |    ├──── app.config.js             # Central config object (reads process.env)
 |    |    ├──── database.config.js        # Sequelize connection options
 |    |    ├──── jwt.config.js             # JWT secret, expiry constants
 |    |    ├──── socket.config.js          # Socket.io server options & CORS
 |    |    ├──── multer.config.js          # File upload storage/filter config
 |    |    ├──── mailer.config.js          # SMTP / email transport (alerts, password reset)
 |    |    └──── logger.config.js          # Winston transports and log format
 |    |
 |    ├──── constants/                     # Immutable application-wide enumerations
 |    |    ├──── roles.js                  # ADMIN | SALES_MANAGER | INVENTORY_MANAGER |
DISPATCH_WORKER
 |    |    ├──── permissions.js            # All permission codes (orders.create,
stock.update …)
 |    |    ├──── orderStatus.js            # DRAFT → CONFIRMED → ALLOCATED → DISPATCHED …
enum
 |    |    ├──── dispatchStatus.js         # QUEUED → PICKING → PACKED → DISPATCHED →
DELIVERED
 |    |    ├──── transactionTypes.js       # PURCHASE_IN | TRANSFER_S1_TO_S2 | DISPATCH_OUT
…
 |    |    ├──── notificationTypes.js      # CREDIT_LIMIT | REORDER | ORDER_STATUS | DISPATCH
…
```

```
│   │   ├── suggestionTypes.js        # REORDER_PRODUCT | UPSELL_CUSTOMER | SLOW_MOVING
…
│   │   ├── auditActions.js           # create | update | delete | approve | login |
export
│   │   ├── importTypes.js            # products | customers | opening_stock
│   │   ├── docTypes.js               # invoice | dispatch_note | receipt | stock_report
…
│   │   └── httpStatus.js             # Named HTTP codes (200 OK, 400, 401, 403, 404,
422, 500)
│   │
│   ├── database/                     # ORM setup and raw DB utilities
│   │   ├── connection.js             # Sequelize instance factory (singleton)
│   │   ├── sync.js                   # Dev-only schema sync helper (never used in prod)
│   │   └── seeders/                  # Initial reference data
│   │       ├── 001-roles.seeder.js
│   │       ├── 002-permissions.seeder.js
│   │       └── 003-role-permissions.seeder.js
│   │
│   ├── models/                       # Sequelize model definitions (one file per table)
│   │   ├── index.js                  # Central model registry + association wiring
│   │   ├── User.js
│   │   ├── Role.js
│   │   ├── Permission.js
│   │   ├── RolePermission.js
│   │   ├── RefreshToken.js
│   │   ├── Customer.js
│   │   ├── Vendor.js
│   │   ├── ProductCategory.js
│   │   ├── Product.js
│   │   ├── Stock1.js
│   │   ├── Stock2.js
│   │   ├── StockTransaction.js
│   │   ├── Order.js
│   │   ├── OrderItem.js
│   │   ├── OrderStatusHistory.js
│   │   ├── Dispatch.js
│   │   ├── DispatchItem.js
│   │   ├── Payment.js
│   │   ├── CreditLimitHistory.js
│   │   ├── ReorderSuggestion.js
│   │   ├── SmartSuggestion.js
```

```
|    |    ├──── Notification.js
|    |    ├──── NotificationRecipient.js
|    |    ├──── AuditLog.js
|    |    ├──── ExcelImportLog.js
|    |    └──── GeneratedDocument.js
|    |
|    ├──── migrations/                    # Sequelize migrations (production schema changes)
|    |    ├──── 20240101000001-create-roles.js
|    |    ├──── 20240101000002-create-users.js
|    |    ├──── 20240101000003-create-permissions.js
|    |    ├──── ... (one per table, strictly ordered)
|    |    └──── 20240101000027-add-indexes.js
|    |
|    ├──── repositories/                  # Data-access layer — all Sequelize queries live
here
|    |    ├──── base.repository.js         # Generic CRUD + soft-delete + pagination helper
|    |    ├──── user.repository.js
|    |    ├──── role.repository.js
|    |    ├──── customer.repository.js
|    |    ├──── vendor.repository.js
|    |    ├──── product.repository.js
|    |    ├──── inventory.repository.js    # Queries spanning stock1, stock2,
stock_transactions
|    |    ├──── order.repository.js
|    |    ├──── orderItem.repository.js
|    |    ├──── dispatch.repository.js
|    |    ├──── payment.repository.js
|    |    ├──── credit.repository.js
|    |    ├──── reorder.repository.js
|    |    ├──── suggestion.repository.js
|    |    ├──── notification.repository.js
|    |    ├──── audit.repository.js
|    |    ├──── excelImport.repository.js
|    |    └──── document.repository.js
|    |
|    ├──── services/                      # Business-logic layer — all rules, no HTTP
awareness
|    |    ├──── auth.service.js            # Login, token issue, refresh, revoke, password
reset
|    |    ├──── user.service.js            # User CRUD, activation toggle
|    |    ├──── role.service.js            # Role & permission matrix management
```

```
│   │   ├── customer.service.js        # Customer CRUD, credit summary aggregation
│   │   ├── vendor.service.js
│   │   ├── product.service.js         # Product CRUD, category tree resolution
│   │   ├── inventory.service.js       # Stock1/Stock2 mutations, available-stock
computation
│   │   ├── stockTransaction.service.js# Ledger writes — the only place stock numbers
change
│   │   ├── order.service.js           # Order lifecycle, credit check, stock reservation
│   │   ├── dispatch.service.js        # Dispatch queue, status transitions, stock-out on
delivery
│   │   ├── payment.service.js         # Payment recording, credit_used reconciliation
│   │   ├── credit.service.js          # Credit limit CRUD, hold/release logic
│   │   ├── reorder.service.js         # Threshold evaluation, suggestion generation
│   │   ├── suggestion.service.js      # Smart suggestion engine — reads, scores,
persists
│   │   ├── notification.service.js    # Create + route notifications to recipients
│   │   ├── audit.service.js           # Audit log writer (called from middleware hook)
│   │   ├── excel.service.js           # Import parsing + export generation via ExcelJS
│   │   ├── pdf.service.js             # All PDF generation via PDFKit
│   │   └── dashboard.service.js       # Per-role aggregated KPI queries
│   │
│   ├── controllers/                   # HTTP layer — parse request, call service, send
response
│   │   ├── auth.controller.js
│   │   ├── user.controller.js
│   │   ├── role.controller.js
│   │   ├── customer.controller.js
│   │   ├── vendor.controller.js
│   │   ├── product.controller.js
│   │   ├── inventory.controller.js
│   │   ├── order.controller.js
│   │   ├── dispatch.controller.js
│   │   ├── payment.controller.js
│   │   ├── credit.controller.js
│   │   ├── reorder.controller.js
│   │   ├── suggestion.controller.js
│   │   ├── notification.controller.js
│   │   ├── audit.controller.js
│   │   ├── excel.controller.js
│   │   ├── pdf.controller.js
│   │   └── dashboard.controller.js
```

```
|    |
|    ├──── routes/                        # Express router definitions
|    |    ├──── index.js                   # Mounts all routers under /api/v1
|    |    ├──── auth.routes.js
|    |    ├──── user.routes.js
|    |    ├──── role.routes.js
|    |    ├──── customer.routes.js
|    |    ├──── vendor.routes.js
|    |    ├──── product.routes.js
|    |    ├──── inventory.routes.js
|    |    ├──── order.routes.js
|    |    ├──── dispatch.routes.js
|    |    ├──── payment.routes.js
|    |    ├──── credit.routes.js
|    |    ├──── reorder.routes.js
|    |    ├──── suggestion.routes.js
|    |    ├──── notification.routes.js
|    |    ├──── audit.routes.js
|    |    ├──── excel.routes.js
|    |    ├──── pdf.routes.js
|    |    └──── dashboard.routes.js
|    |
|    ├──── middleware/                    # Express middleware (request pipeline)
|    |    ├──── authenticate.js            # JWT verification → req.user
|    |    ├──── authorize.js               # Permission code check factory
|    |    ├──── rateLimiter.js             # express-rate-limit configs (global + auth-
specific)
|    |    ├──── validate.js                # Runs express-validator chains, collects errors
|    |    ├──── auditHook.js               # Post-response middleware — writes audit log
entries
|    |    ├──── errorHandler.js            # Global error handler (last middleware in chain)
|    |    ├──── notFound.js                # 404 catcher (after all routes)
|    |    ├──── requestLogger.js           # Per-request Morgan + Winston logging
|    |    ├──── uploadHandler.js           # Multer instance wrapper for file routes
|    |    └──── sanitizer.js              # xss-clean / input scrubbing
|    |
|    ├──── validators/                    # express-validator chain definitions (no logic,
only rules)
|    |    ├──── auth.validator.js
|    |    ├──── user.validator.js
|    |    ├──── customer.validator.js
```

```
│   │   ├── vendor.validator.js
│   │   ├── product.validator.js
│   │   ├── inventory.validator.js
│   │   ├── order.validator.js
│   │   ├── dispatch.validator.js
│   │   ├── payment.validator.js
│   │   ├── credit.validator.js
│   │   ├── reorder.validator.js
│   │   ├── excel.validator.js
│   │   └── pagination.validator.js    # Reusable page/limit/sort chains
│   │
│   ├── sockets/                       # Socket.io server-side event architecture
│   │   ├── index.js                    # Socket server init, auth handshake, room join
│   │   ├── handlers/
│   │   │   ├── order.handler.js        # order:updated, order:status_changed events
│   │   │   ├── dispatch.handler.js     # dispatch:assigned, dispatch:status_changed
│   │   │   ├── notification.handler.js# notification:new broadcast
│   │   │   ├── inventory.handler.js    # stock:low_alert, stock:reorder_triggered
│   │   │   └── credit.handler.js       # credit:limit_breach, credit:hold_placed
│   │   ├── rooms.js                    # Room name generators (role:admin, user:42,
order:99)
│   │   └── emitter.js                  # Singleton emitter used by services to push
events
│   │
│   ├── jobs/                          # Cron / scheduled background tasks
│   │   ├── scheduler.js                # node-cron job registry and startup
│   │   ├── reorderCheck.job.js         # Runs nightly — compares stock1 vs thresholds
│   │   ├── creditSweep.job.js          # Hourly — recalculates credit_used across open
orders
│   │   ├── suggestionRefresh.job.js    # Daily — regenerates smart suggestions
│   │   └── tokenCleanup.job.js         # Daily — purges expired refresh_tokens
│   │
│   ├── utils/                         # Pure helper functions (no side effects, no DB
calls)
│   │   ├── response.util.js            # Standardised API response shape { success, data,
meta, error }
│   │   ├── pagination.util.js          # Page/offset/limit/totalPages calculator
│   │   ├── hash.util.js                # bcrypt wrappers
│   │   ├── token.util.js               # JWT sign / verify / decode helpers
│   │   ├── date.util.js                # Date formatting, business-day calculations
│   │   ├── number.util.js              # Currency rounding, percentage helpers
```

```
│   │   ├── string.util.js            # Slug generation, code generation (ORD-XXXXX)
│   │   ├── file.util.js              # File path helpers, temp-file cleanup
│   │   ├── excel.util.js             # Row-to-model mapping helpers for import
│   │   └── crypto.util.js            # Random token generation for refresh tokens
│   │
│   ├── errors/                       # Custom error class hierarchy
│   │   ├── AppError.js               # Base error (message, statusCode, isOperational)
│   │   ├── ValidationError.js        # 422 — field-level validation failures
│   │   ├── AuthenticationError.js    # 401 — invalid/missing credentials
│   │   ├── AuthorizationError.js     # 403 — valid user, insufficient permission
│   │   ├── NotFoundError.js          # 404 — entity not found
│   │   ├── ConflictError.js          # 409 — duplicate key, state conflict
│   │   ├── BusinessRuleError.js      # 400 — credit hold, insufficient stock, etc.
│   │   └── ExternalServiceError.js   # 502 — upstream failure (mailer, storage)
│   │
│   ├── logs/                         # Runtime log output (gitignored)
│   │   ├── combined.log
│   │   ├── error.log
│   │   └── audit.log
│   │
│   ├── uploads/                      # Multer temp storage (gitignored, cleaned by job)
│   │
│   ├── generated/                    # PDFs and Excel exports (gitignored, path stored
in DB)
│   │   ├── invoices/
│   │   ├── dispatch-notes/
│   │   ├── receipts/
│   │   └── reports/
│   │
│   └── app.js                        # Express app factory (no listen — keeps testable)
│
├── server.js                         # Entry point: creates app, attaches Socket.io,
calls listen()
├── .env                              # Environment variables (gitignored)
├── .env.example                      # Template for all required variables
├── .sequelizerc                      # Points sequelize-cli to src/database & src/models
├── .eslintrc.js
├── .prettierrc
├── jest.config.js
├── package.json
└── README.md
```

---

2. API Architecture

Base URL

All routes are mounted at `/api/v1`.

Response Envelope

Every response — success and error — uses the same shape, produced by `response.util.js`:

```
{
  "success": true | false,
  "data": { ... } | [ ... ] | null,
  "meta": { "page": 1, "limit": 20, "total": 340, "totalPages": 17 },
  "error": { "code": "VALIDATION_ERROR", "message": "...", "fields": [...] } | null,
  "requestId": "uuid-for-tracing"
}
```

Route Table (all 17 modules)

Domain	Method	Path	Permission
Auth	POST	/auth/login	public
	POST	/auth/refresh	public (refresh token)
	POST	/auth/logout	authenticated
	POST	/auth/forgot-password	public
	POST	/auth/reset-password	public (reset token)
Users	GET	/users	users.read
	POST	/users	users.create
	GET	/users/:id	users.read
	PATCH	/users/:id	users.update

	PATCH	/users/:id/status	users.update
Roles	GET	/roles	roles.read
	GET	/roles/:id/permissions	roles.read
	PUT	/roles/:id/permissions	roles.update
Dashboard	GET	/dashboard	dashboard.read
Customers	GET	/customers	customers.read
	POST	/customers	customers.create
	GET	/customers/:id	customers.read
	PATCH	/customers/:id	customers.update
	DELETE	/customers/:id	customers.delete
	GET	/customers/:id/orders	orders.read
	GET	/customers/:id/credit	credit.read
Vendors	GET	/vendors	vendors.read
	POST	/vendors	vendors.create
	GET	/vendors/:id	vendors.read
	PATCH	/vendors/:id	vendors.update
Products	GET	/products	products.read
	POST	/products	products.create
	GET	/products/:id	products.read
	PATCH	/products/:id	products.update
	DELETE	/products/:id	products.delete
	GET	/categories	products.read
Inventory	GET	/inventory	inventory.read

	GET	/inventory/:productId	inventory.read
	POST	/inventory/adjustment	inventory.update
	GET	/inventory/transactions	inventory.read
Orders	GET	/orders	orders.read
	POST	/orders	orders.create
	GET	/orders/:id	orders.read
	PATCH	/orders/:id	orders.update
	POST	/orders/:id/confirm	orders.approve
	POST	/orders/:id/cancel	orders.update
	POST	/orders/:id/hold	orders.approve (admin)
Dispatch	GET	/dispatches	dispatch.read
	GET	/dispatches/queue	dispatch.read
	GET	/dispatches/:id	dispatch.read
	PATCH	/dispatches/:id/status	dispatch.update
	POST	/dispatches/:id/deliver	dispatch.update
Payments	GET	/payments	payments.read
	POST	/payments	payments.create
	GET	/payments/:id	payments.read
Credit	GET	/credit	credit.read
	PATCH	/credit/:customerId	credit.update
	GET	/credit/:customerId/history	credit.read
Reorder	GET	/reorder	reorder.read
	PATCH	/reorder/:id/acknowledge	reorder.update

	PATCH	/reorder/:id/convert	reorder.update
	PATCH	/reorder/:id/dismiss	reorder.update
	PATCH	/products/:id/threshold	reorder.update
Suggestions	GET	/suggestions	suggestions.read
	PATCH	/suggestions/:id/act	suggestions.update
	PATCH	/suggestions/:id/dismiss	suggestions.update
Notifications	GET	/notifications	notifications.read
	PATCH	/notifications/:id/read	notifications.update
	POST	/notifications/mark-all-read	notifications.update
Audit	GET	/audit	audit.read
	GET	/audit/:id	audit.read
Excel	POST	/excel/import	excel.import
	GET	/excel/export/:type	excel.export
	GET	/excel/import/:id/status	excel.import
PDF	GET	/pdf/invoice/:orderId	pdf.generate
	GET	/pdf/dispatch-note/:dispatchId	pdf.generate
	GET	/pdf/receipt/:paymentId	pdf.generate
	GET	/pdf/stock-report	pdf.generate
	GET	/pdf/audit-report	pdf.generate

---

3. Middleware Architecture

The request pipeline is assembled in `app.js` in strict order. Every incoming request passes through layers left to right before reaching a controller.

```
Request
   |
   ├── 1. Security headers      helmet()
   ├── 2. CORS                  cors() with whitelist from config
   ├── 3. Request logger        requestLogger.js (Morgan format → Winston)
   ├── 4. Body parser           express.json({ limit: '10mb' })
   ├── 5. Input sanitiser       sanitizer.js (xss-clean)
   ├── 6. Global rate limiter   rateLimiter.js (100 req/15 min per IP)
   |
   ├── [route matched]
   |
   ├── 7. Auth rate limiter     rateLimiter.js (10 req/15 min — login routes only)
   ├── 8. authenticate.js       Verifies JWT → attaches req.user { id, role, permissions }
   ├── 9. authorize(code)       Checks req.user.permissions includes required code
   ├── 10. uploadHandler.js     Multer (file routes only)
   ├── 11. validate(chains)     express-validator → throws ValidationError on failure
   |
   ├── [controller called]
   |
   ├── 12. auditHook.js         Post-response hook — writes AuditLog for mutating methods
   ├── 13. notFound.js          Catches unmatched routes → 404
   └── 14. errorHandler.js      Global catch — formats AppError subclasses to envelope
```

Middleware File Responsibilities

authenticate.js

Extracts the Bearer token from `Authorization` header. Calls `token.util.js` to verify signature and expiry. Checks the token's `jti` is not in the revoked set (stored in the `refresh_tokens` table on logout). Attaches the decoded payload as `req.user`. Throws `AuthenticationError` if any check fails.

authorize.js

Factory function — `authorize('orders.create')` returns an Express middleware that reads `req.user.permissions` (an array of permission codes loaded at login) and throws `AuthorizationError` if the required code is absent. The permission array is embedded in the JWT at login time, so no DB hit per request.

rateLimiter.js

Exports two pre-configured `express-rate-limit` instances: `globalLimiter` (applied to all routes) and `authLimiter` (stricter, applied only to `/auth/login` and `/auth/refresh`). Uses an in-memory store (upgradeable to Redis for multi-process deployments).

validate.js

Takes an array of `express-validator` chain objects. Runs them, collects `validationResult`, and if errors exist throws a `ValidationError` with the structured field errors array. Controllers never see invalid input.

auditHook.js

Attached as `res.on('finish', ...)`. Only fires for mutating HTTP methods (POST, PATCH, PUT, DELETE). Reads `req.auditContext` (set by controllers before sending responses) which contains `{ module, entityType, entityId, beforeState, afterState }`. Calls `audit.service.js` asynchronously — the response is never delayed by audit writes.

errorHandler.js

The final `(err, req, res, next)` middleware. Checks `err instanceof AppError`. Operational errors are formatted into the response envelope. Unknown errors are logged as critical and return a generic 500. Sequelize `UniqueConstraintError` and `ValidationError` are mapped to `ConflictError` and `ValidationError` respectively before reaching this handler.

---

4. Service Architecture

Services are the heart of the system. They own all business logic. They know nothing about HTTP — they receive plain arguments and return plain objects or throw `AppError` subclasses.

Cross-Cutting Service Rules

- Every service method that mutates data must call `stockTransaction.service.js` or a domain-equivalent ledger — never raw model updates without an audit trail.
- Services call repositories for data access. They never call Sequelize models directly.
- Services call `notification.service.js` to fire alerts. Notification delivery is non-blocking.
- Services call `socket.emitter.js` to push real-time events after state changes.

Key Service Logic Summaries

auth.service.js

`login(email, password)` — loads user+role, verifies bcrypt hash, loads permission codes via role, signs access JWT (15 min) and refresh JWT (7 days), stores refresh token hash. `refresh(token)` — verifies refresh token, checks it is not revoked, issues new access token. `logout(userId, tokenJti)` — revokes the refresh token. `forgotPassword(email)` — generates a time-limited reset token, sends email. `resetPassword(token, newPassword)` — validates token, hashes password, invalidates all active refresh tokens for the user.

inventory.service.js

`getAvailableStock(productId)` — reads `stock1.quantity_on_hand - stock2.quantity_reserved`. This is the only authoritative "can we sell this?" answer. `reserveStock(productId, qty, orderId)` — transactional: decrements available stock by writing a `TRANSFER_S1_TO_S2` ledger entry and incrementing `stock2`. `releaseReservation(productId, qty, orderId)` — reversal on order cancellation. `finaliseDispatch(productId, qty, dispatchId)` — writes `DISPATCH_OUT` entry, decrements both `stock1` and `stock2`. Triggers reorder check inline if new `stock1` drops below `reorder_threshold`.

order.service.js

`createOrder(data, salesManagerId)` — validates customer exists and is assigned to this manager, checks credit availability (`customer.credit_limit_amount - credit_used_amount >= order_total`), creates order in DRAFT status, writes `order_status_history`. `confirmOrder(orderId, userId)` — transitions DRAFT → CONFIRMED, calls `inventory.service.reserveStock` for every line item atomically (Sequelize transaction), transitions to ALLOCATED on success or sets `credit_hold = true` on credit failure. `cancelOrder(orderId, userId, reason)` — releases all stock reservations, transitions to CANCELLED, fires notification.

payment.service.js

`recordPayment(data, userId)` — creates payment record, recalculates `customer.credit_used_amount` by summing all unpaid confirmed orders minus payments received, updates the customer row atomically. If the credit hold was triggered by the paid invoice, calls `credit.service.releaseHold`.

suggestion.service.js

Runs on a cron schedule and on-demand. Queries: slow-moving products (no order lines in last 30 days), customers due for repeat orders (previous order cycles), reorder products. Scores each suggestion by a priority formula, upserts into `smart_suggestions`. Does not send notifications directly — notifiers read the suggestions table.

---

5. Controller Architecture

Controllers are intentionally thin. A controller method does exactly four things in order:

1. Extract and type-cast parameters from `req` (params, query, body, user, file).
2. Call one service method.
3. Set `req.auditContext` if the operation is auditable.
4. Send the response via `response.util.js`.

No business logic, no DB calls, no direct model access. If a controller method exceeds ~30 lines, logic belongs in the service layer.

Controller Pattern Example (all controllers follow this shape)

```
GET /orders/:id
  → extract: { id } from req.params, req.user
  → call: order.service.getOrderById(id, req.user)
  → send: res.json(successResponse(order))

POST /orders
  → extract: body fields + req.user.id
  → call: order.service.createOrder(body, req.user.id)
  → set: req.auditContext = { module: 'orders', entityType: 'Order', entityId: order.id,
afterState: order }
  → send: res.status(201).json(successResponse(order))
```

---

6. Repository Architecture

Repositories are the only layer that imports Sequelize models. They own all query construction — `WHERE`, `INCLUDE`, `ORDER`, `LIMIT`, `OFFSET`, aggregations.

base.repository.js

Provides generic methods reused by all domain repositories:

- `findAll(model, { where, include, order, page, limit })` — returns `{ rows, count }` with pagination metadata.
- `findById(model, id, include)` — throws `NotFoundError` if null.
- `findOne(model, where)` — returns null on miss, does not throw.
- `create(model, data, transaction)` — wraps model.create.
- `update(model, id, data, transaction)` — wraps model.update, returns updated instance.
- `softDelete(model, id, transaction)` — sets `deleted_at`.
- `hardDelete(model, id, transaction)` — for non-master-data only.

Domain Repositories

Each domain repository extends or composes `base.repository.js` and adds domain-specific queries. Examples:

inventory.repository.js
`getStockSummary(productId)` — joins `products`, `stock1`, `stock2` in one query to return the full stock picture. `getLowStockProducts()` — `WHERE stock1.quantity_on_hand <= products.reorder_threshold`. `getTransactionLedger(productId, dateRange)` — paginated ledger with actor join.

order.repository.js
`getOrdersForSalesManager(managerId, filters)` — scopes to `sales_manager_id` automatically. `getOrderWithItems(orderId)` — deep include: customer, orderItems → product. `getOrdersByStatus(status, page, limit)`.

audit.repository.js
`search(filters)` — supports complex filtering on `module`, `entity_type`, `actor_id`, `created_at` range. All queries are indexed.

---

7. Socket Architecture

Initialization (`sockets/index.js`)

The Socket.io server is attached to the HTTP server in `server.js`. On connection:

1. Middleware extracts the JWT from `handshake.auth.token`.
2. Verifies token with `token.util.js`. Invalid tokens get `socket.disconnect()`.
3. Successful auth → `socket.data.user` is set.
4. The socket is joined to its rooms automatically: `role:{roleName}` (e.g. `role:sales_manager`), `user:{userId}`, and for dispatch workers: `dispatch:{workerId}`.

Room Strategy

Room	Joined by	Purpose
`role:admin`	Admin users	System-wide alerts, all order events
`role:sales_manager`	All SMs	Credit alerts, new suggestions
`role:inventory_manager`	All IMs	Stock alerts, reorder triggers
`role:dispatch_worker`	All DWs	Queue updates
`user:{id}`	Each user	Personal notifications
`order:{id}`	SM who owns order + IM + DW assigned	Live order status updates

Emitter (`sockets/emitter.js`)

A singleton that wraps the Socket.io `io` instance. Exported and imported by services. Services call `emitter.toRoom(room, event, payload)` — this decouples services from the socket layer. Services never import `io` directly.

Event Catalog

Event	Emitted by	Received by
`order:status_changed`	order.service	order room members
`dispatch:assigned`	dispatch.service	`role:dispatch_worker`
`dispatch:status_changed`	dispatch.service	order room, `role:admin`
`stock:low_alert`	inventory.service	`role:inventory_manager`, `role:admin`
`credit:hold_placed`	order.service	`user:{smId}`, `role:admin`
`notification:new`	notification.service	`user:{recipientId}` or `role:{roleName}`
`reorder:triggered`	reorderCheck.job	`role:inventory_manager`

---

8. Validation Architecture

All input validation uses `express-validator`. Validators are pure arrays of chain objects — no logic, no service calls.

Validator File Structure

Each file in `/validators` exports named arrays consumed by routes:

js

```
// order.validator.js
export const createOrderRules = [
  body('customer_id').isInt({ min: 1 }).withMessage('Valid customer_id required'),
  body('items').isArray({ min: 1 }).withMessage('At least one item required'),
  body('items.*.product_id').isInt({ min: 1 }),
  body('items.*.quantity').isInt({ min: 1 }).withMessage('Quantity must be >= 1'),
  body('items.*.unit_price').isDecimal({ decimal_digits: '0,2' })
]
```

Route Usage Pattern

js

```
// order.routes.js
router.post('/', authenticate, authorize('orders.create'),
  createOrderRules, validate, order.controller.create)
```

`validate` middleware (from `middleware/validate.js`) runs `validationResult(req)`, collects errors, and throws `ValidationError({ fields: [...] })` before the controller is ever reached.

Pagination Validator

`pagination.validator.js` exports a reusable `paginationRules` array (page, limit, sortBy, sortDir) that is spread into any listing route's validator array.

---

9. Logging Architecture

Logging uses **Winston** for structured logs and **Morgan** for HTTP access logs.

Winston Transports

Transport	File	Level	Condition
Console	—	debug	Development only
File	`logs/combined.log`	info	All environments
File	`logs/error.log`	error	All environments
File	`logs/audit.log`	info	Audit entries only (separate stream)

Log Levels Used

`error` — unhandled exceptions, DB connection failures, external service errors.

`warn` — JWT near-expiry, rate limit approaches, deprecated API usage.

`info` — request completed, cron job ran, socket room joined.

`debug` — query details, service method args (development only, never in production).

Structured Log Shape

Every log entry is JSON:

json

```
{
  "timestamp": "2024-01-15T10:23:45.123Z",
  "level": "info",
  "requestId": "uuid",
  "userId": 42,
  "role": "sales_manager",
  "module": "orders",
  "message": "Order ORD-00123 confirmed",
  "duration_ms": 34
}
```

requestLogger.js

Morgan generates HTTP access log lines in a custom format and pipes them through a Winston `stream`. This keeps all log output in one place.

---

10. Audit Architecture

Audit logging is mandatory for every state-changing operation. It is implemented as a post-response hook, not as service-layer calls, so it cannot accidentally be forgotten when a new route is added.

Flow

1. Controller calls service, gets result.
2. Controller sets `req.auditContext = { module, entityType, entityId, beforeState, afterState, actionType }`. The `beforeState` is fetched by the service before mutation and passed back as part of the result object.
3. Controller sends HTTP response.

4. `auditHook.js` fires on `res.on('finish')`. Only for methods POST/PATCH/PUT/DELETE. Reads `req.auditContext` and calls `audit.service.writeLog(...)`.
5. `audit.service.js` calls `audit.repository.create(...)` with actor details from `req.user`, IP from `req.ip`, and the context object.
6. Write is asynchronous — no `await` in the hook. The response is never delayed.

AuditLog Record Shape

```
{
  actor_id, actor_role,
  action_type,           -- create | update | delete | approve | login | export
  module,                -- 'orders' | 'inventory' | 'customers' …
  entity_type,           -- 'Order' | 'Stock1' | 'Customer' …
  entity_id,             -- PK of the affected record
  before_state,          -- JSON snapshot before change (null for creates)
  after_state,           -- JSON snapshot after change (null for deletes)
  ip_address,
  created_at
}
```

Admin Audit Query API

`GET /audit` supports filtering by `module`, `entity_type`, `actor_id`, `action_type`, date range, and full-text search on `before_state` / `after_state` (MySQL JSON functions). Results are paginated and exportable to PDF via `GET /pdf/audit-report`.

---

11. Notification Architecture

Notifications are persisted in the database AND pushed in real-time via Socket.io. This two-channel approach ensures delivery even if the user is offline.

Notification Creation Flow

1. A service (e.g. `order.service`) calls `notification.service.create({ type, title, message, severity, payload, recipients })`.

2. `notification.service` writes a `notifications` row and one `notification_recipients` row per recipient (by user ID or role ID for broadcasts).
3. After DB write, calls `emitter.toRoom(...)` with the `notification:new` event.
4. Frontend receives the event and increments the bell counter without a page reload.

Recipient Resolution

`recipients` can be specified as:

- `{ userId: 42 }` — personal notification.
- `{ roleId: 2 }` — broadcast to all active users of that role.
- `[ { userId: 1 }, { roleId: 3 } ]` — mixed.

The service resolves role broadcasts into individual `notification_recipients` rows so each user's read state is tracked independently.

Notification Categories and Default Routing

Type	Severity	Default Recipients
`CREDIT_LIMIT_BREACH`	critical	SM who owns order, Admin
`STOCK_LOW`	warning	Inventory Manager, Admin
`REORDER_TRIGGERED`	info	Inventory Manager
`ORDER_CONFIRMED`	info	Inventory Manager
`ORDER_CANCELLED`	warning	SM, Inventory Manager
`DISPATCH_ASSIGNED`	info	Dispatch Worker
`DELIVERY_CONFIRMED`	info	SM, Admin
`CREDIT_HOLD_RELEASED`	info	SM who owns order

`SMART_SUGGESTION_NEW`	info	SM or IM (per suggestion target role)

---

12. PDF Architecture

PDF generation uses **PDFKit** and is always asynchronous — the client gets a download URL, not a streamed buffer.

Generation Flow

1. `GET /pdf/invoice/:orderId` controller is called.
2. Permission checked (`pdf.generate`).
3. Controller calls `pdf.service.generateInvoice(orderId, requestingUserId)` .
4. Service fetches all required data via repositories (order, items, customer, payments).
5. Service calls the relevant PDFKit builder function (see below).
6. Output is piped to a file at `src/generated/invoices/{orderId}_{timestamp}.pdf` .
7. File path is stored in `generated_documents` table.
8. Service returns `{ fileUrl: '/downloads/invoices/...', documentId }` .
9. Controller returns `{ fileUrl }` to client.
10. Client opens the URL to download.

PDF Builder Functions (`pdf.service.js`)

Each document type has its own builder function that takes a data object and a PDFKit `doc` instance:

- `buildInvoice(doc, { order, customer, items, payments })` — letterhead, order header table, line items table, tax calculation block, payment summary, signature area.
- `buildDispatchNote(doc, { dispatch, order, items, worker })` — picklist table with checkboxes, delivery address, QR code area.
- `buildReceipt(doc, { payment, customer, order })` — single-page receipt.
- `buildStockReport(doc, { products, stock1, stock2, dateRange })` — multi-page table with reorder highlights.

- `buildAuditReport(doc, { logs, filters, generatedBy })` — filtered audit trail with actor, action, timestamp columns.

File Management

Generated files are stored in `src/generated/{type}/`. A cleanup cron job (configurable, default 30 days) deletes old files. Paths in `generated_documents` are retained as a record. File access is gated behind the same JWT auth — files are served via an authenticated Express static route, not directly exposed.

---

13. Excel Architecture

Excel import and export use **ExcelJS**. Import uses **Multer** for file upload.

Import Flow

1. `POST /excel/import` with `multipart/form-data` carrying the file and `import_type`.
2. `uploadHandler.js` (Multer) saves the file to `src/uploads/`.
3. `excel.controller.importFile` calls `excel.service.processImport(filePath, importType, userId)`.
4. Service creates an `excel_import_logs` row with status `PROCESSING` and returns `{ importLogId }` immediately (async processing).
5. A background worker (or async function) reads the workbook row by row using ExcelJS streaming reader.
6. Each row is validated against the import type's schema (`excel.util.rowToModel`).
7. Valid rows are bulk-inserted via repository. Invalid rows are collected with their row numbers and error messages.
8. On completion, `excel_import_logs` is updated: `status = COMPLETED`, `success_rows`, `failed_rows`, `error_report_path` (a JSON file listing row errors).
9. Client polls `GET /excel/import/:id/status` for progress.

Export Flow

1. `GET /excel/export/:type?filters=...` is called.

2. `excel.service.generateExport(type, filters, userId)` runs.
3. ExcelJS `Workbook` is created in memory. Worksheet is populated from repository query.
4. Workbook is written to `src/generated/reports/{type}_{timestamp}.xlsx`.
5. URL returned to client.

Import Templates

Each `importType` has a defined column schema in `excel.util.js`:

Import Type	Expected Columns
`products`	SKU, Name, Category, Unit, Unit Price, Cost Price, Reorder Threshold, Reorder Qty
`customers`	Code, Name, Phone, Email, GST Number, Credit Limit, Payment Terms Days
`opening_stock`	SKU, Quantity On Hand

Column names are matched case-insensitively. Missing required columns trigger a pre-processing error that aborts the import before any rows are written.

---

14. Security Architecture

Authentication

- Access tokens: JWT, HS256, 15-minute expiry. Payload: `{ sub: userId, role, permissions[], jti }`.
- Refresh tokens: opaque random string, hashed with SHA-256 before storage, 7-day expiry.
- Logout revokes the refresh token. Access token revocation relies on short expiry (no per-request DB check needed at scale).
- Password hashing: bcrypt with cost factor 12.

Authorization

- RBAC enforced server-side by `authorize.js` middleware using permission codes from JWT payload.
- Row-level scoping is enforced inside repositories — e.g., Sales Manager queries are always filtered by `sales_manager_id = req.user.id`.
- Admin override paths (e.g., force-cancel an order) use a separate permission code (`orders.admin_override`) checked explicitly.

Transport Security

- HTTPS enforced by Nginx reverse proxy (TLS termination).
- HSTS header set via Helmet.
- CORS restricted to the known frontend origins in config.

Input Security

- All user input is sanitised by `sanitizer.js` (XSS-clean) before reaching controllers.
- SQL injection is prevented by Sequelize parameterised queries — raw queries are forbidden.
- File uploads: MIME type and extension are validated by Multer filter. Maximum file size enforced (10MB). Files stored outside the web root.

Rate Limiting

- Global: 100 requests per 15 minutes per IP.
- Auth routes: 10 requests per 15 minutes per IP (brute-force protection).

Secrets Management

- All secrets in `.env`, never hardcoded.
- `.env` is gitignored. `.env.example` documents required variables without values.
- In production, environment variables are injected by the deployment platform (not file-based).

---

15. Database Connection Architecture

Connection (`database/connection.js`)

A single Sequelize instance is created as a module-level singleton. It is the only place connection options are read from `database.config.js`. The instance is imported by `models/index.js` and `repositories` — never re-instantiated.

Connection Pool Configuration

```
pool: {
  max: 10,          // Maximum connections in pool
  min: 2,           // Minimum idle connections kept alive
  acquire: 30000,   // Max ms to wait for a connection before throwing
  idle: 10000       // Max ms a connection can be idle before release
}
```

Connection Health

On server startup, `server.js` calls `sequelize.authenticate()`. If it fails, the process exits with a non-zero code — the process manager (PM2) restarts it and alerts on repeated failure.

Transactions

All multi-step mutations (e.g., confirm order = reserve stock for N items + update order status) use a Sequelize `transaction`. The transaction object is passed through the service method signature into every repository call involved in the operation. On any failure, the entire transaction rolls back automatically.

Pattern: services call `sequelize.transaction(async (t) => { ... })`. Repositories accept an optional `{ transaction: t }` options argument.

Migrations

Schema changes use Sequelize CLI migrations (`sequelize-cli`). Production deployments run `npx sequelize db:migrate` before starting the app. There are no auto-sync calls in production. Migration files are numbered sequentially and never modified after they have been run.

---

16. Environment Architecture

`.env.example` — All Required Variables

ini

```
# Application
NODE_ENV=development
PORT=3000
APP_URL=http://localhost:3000
CLIENT_URL=http://localhost:5173

# Database
DB_HOST=localhost
DB_PORT=3306
DB_NAME=erp_db
DB_USER=erp_user
DB_PASS=change_me_in_prod

# JWT
JWT_ACCESS_SECRET=replace_with_64_char_random_string
JWT_REFRESH_SECRET=replace_with_64_char_random_string
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d

# Rate Limiting
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX=100
AUTH_RATE_LIMIT_MAX=10

# File Storage
UPLOAD_DIR=src/uploads
GENERATED_DIR=src/generated
MAX_FILE_SIZE_MB=10

# Email (SMTP)
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=noreply@erp.com
SMTP_PASS=change_me
SMTP_FROM="ERP System <noreply@erp.com>"
```

```
# Logging
LOG_LEVEL=info
LOG_DIR=src/logs

# Cron Schedules (cron syntax)
REORDER_CHECK_SCHEDULE=0 2 * * *
CREDIT_SWEEP_SCHEDULE=0 * * * *
SUGGESTION_REFRESH_SCHEDULE=0 3 * * *
TOKEN_CLEANUP_SCHEDULE=0 4 * * *
```

Config Object (`config/app.config.js`)

All `process.env` reads happen once in this file. The rest of the codebase imports from this config object — never directly from `process.env`. This centralises validation (missing required vars throw on startup) and makes the config testable.

---

17. Cron Job Architecture

All scheduled jobs use **node-cron**. They are registered in `jobs/scheduler.js` which is called once from `server.js` after the DB connection is confirmed.

Each job file exports a single async function. The scheduler wraps each in a try/catch with full Winston error logging. Jobs are designed to be idempotent — re-running a job that already ran should produce the same DB state as running it once.

Job	Schedule	What it does
`reorderCheck.job.js`	02:00 daily	Queries all products where `stock1.qty <= reorder_threshold`. Creates `reorder_suggestions` rows for any without an open `NEW` suggestion. Fires `reorder:triggered` socket event.

`creditSweep.job.js`	Every hour	Recalculates `credit_used_amount` for all customers with open orders. Updates customer rows. Fires `credit:hold_placed` for any newly breached limits.
`suggestionRefresh.job.js`	03:00 daily	Deletes stale `smart_suggestions` (>7 days old, status NEW). Runs scoring logic for all active sales managers and inventory managers. Inserts fresh suggestions.
`tokenCleanup.job.js`	04:00 daily	Hard-deletes `refresh_tokens` where `expires_at < NOW()` or `revoked_at IS NOT NULL AND revoked_at < 30 days ago`.

Architecture Decision Record (Key Choices)

Why repositories instead of direct model calls in services?
Isolates query logic so services stay readable. Makes testing services with mock repositories trivial. Enables query optimisations (adding indexes, query hints) without touching business logic.

Why embed permissions in the JWT?
Eliminates a DB round-trip on every request. Permissions change rarely. A 15-minute token expiry limits the staleness window. An admin changing a role takes effect within 15 minutes without any cache invalidation complexity.

Why async audit writes?
Audit logging must never slow down a business transaction. A failed audit write is logged to the error file but does not roll back the operation. Auditability is important but is not a reason to degrade user-facing latency.

Why two stock tables (stock1/stock2) instead of one?

Stock reservation (S2) must be decoupled from physical stock (S1) so "available to sell" can be calculated accurately without scanning open orders. The `stock_transactions` ledger is the true source of truth; stock1 and stock2 are cached derived totals for fast reads.

Why store generated PDFs as files rather than streaming?

Large report PDFs can take several seconds to generate. Returning a file URL lets the client initiate download on demand and lets the server generate asynchronously. File paths in the `generated_documents` table provide an audit trail of every document ever produced.